Final Project Submission

DeepCompile: AI-Driven Compiler Optimization

Roll Number: 24CSB0A06 · Date: April 19, 2026

Final Project Submission: DeepCompile

AI-Driven Compiler Optimization with Formal Verification

Roll Number: 24CSB0A06

Module: CD Refined

Date: April 19, 2026

1. Project Overview

DeepCompile is an end-to-end multi-agent system designed for safe, verified, and security-audited C/C++ code optimization. It addresses the reliability gaps in modern LLM-based code assistants by wrapping them in a rigorous formal verification and security auditing framework.

Core Objectives:

- **Correctness:** Ensuring optimized code is semantically identical to the original.
- **Security:** Preventing the introduction of new vulnerabilities during transformation.
- **Performance:** Measurable speedups via rule-based and LLM-driven techniques.
- **Explainability:** Providing detailed rationale for every optimization applied.

2. System Architecture

The system is built on a modular multi-agent framework where each agent specializes in a specific stage of the compilation life-cycle.

Key Components:

1. **Analysis Agent:** Uses a regex-based `CodeParser` to identify bugs (uninitialized variables, memory leaks) and optimization opportunities.

2. **Optimization Agent:** Applies a hybrid approach of rule-based fixes and LLM-generated transformations using Qwen 2.5 Coder 7B.

3. **Verification Agent:** Implements a 4-layer check:

- **Differential Testing:** Execution output comparison.

- **Z3 SMT Prover:** Formal logical equivalence proof.

- **Performance Benchmark:** Nanosecond-level timing via `std::chrono`.

- **LLM Reasoning:** Semantic equivalence assessment.

4. **Security Agent:** Performs a 4-layered audit (Rules, Heuristics, LLM, cppcheck) to ensure zero new vulnerabilities.

---

3. Results and Performance

The system was evaluated against the `MicroBenchmarks` suite, achieving a high success rate and significant performance gains.



Summary Metrics:

- **Pass Rate:** 83.3% on benchmark cases.
- **Security:** Zero (0) new vulnerabilities introduced.
- **Rollback Effectiveness:** Successfully reverted 100% of cases that failed verification or security audits.

---

4. Key Features

- **Safe-by-Design:** Automatic context rollback on any verification failure.
- **Multi-Backend LLM:** Support for Ollama (local) and Google Gemini (cloud).
- **Explainable AI:** Every change is accompanied by a Chain-of-Thought (CoT) reasoning report.
- **Real-time Monitoring:** React-based UI with SSE streaming for live pipeline visibility.

---

5. Conclusion

DeepCompile demonstrates that integrating LLMs into the compiler toolchain is viable and safe when combined with traditional formal methods and static analysis. The system provides a robust blueprint for future AI-assisted development tools that prioritize safety and correctness alongside performance.